Coroutines

Lieven de Cock
www.codeblocks.org

lieven@cppdriven.com

About me

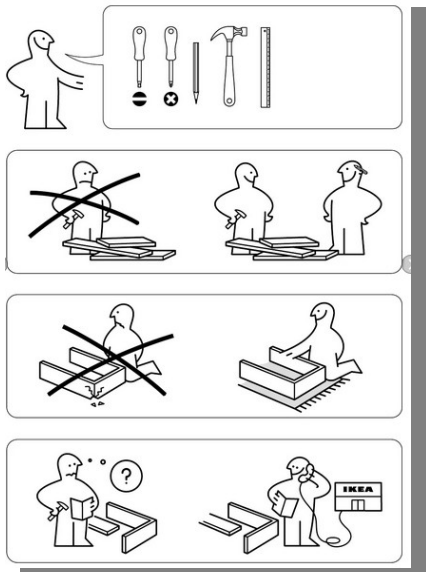
- 30+ years experience in C++ development (speech synthesis/recognition, video detection, satellite communication, ...)
- Many roles: developer, architect, manager, coach, …
- Contributor to several open source projects
- Independent consultant/coach/trainer for C++ and it's eco-system → lieven@cppdriven.com
- Clean code evangelist, always raise the bar on quality (apply the eco-system)

What we want



What we got





Myths

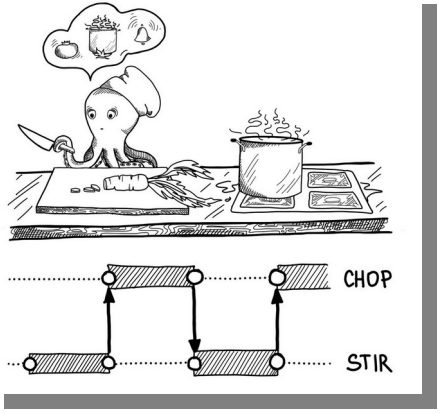
- Is NOT related to:
  - Multi threading
  - Asynchronous programming

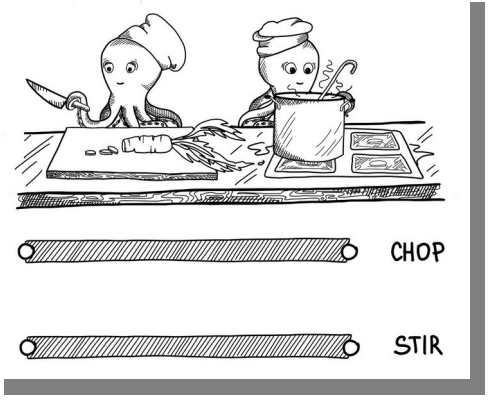
- It can be used in those areas, like it can be used in other areas

Concurrency versus Parallelism

- Processes
- Threads
- Coroutines

- A system is said to be concurrent if it can support two or more actions in progress at the same time. A system is said to be parallel if it can support two or more actions executing simultaneously

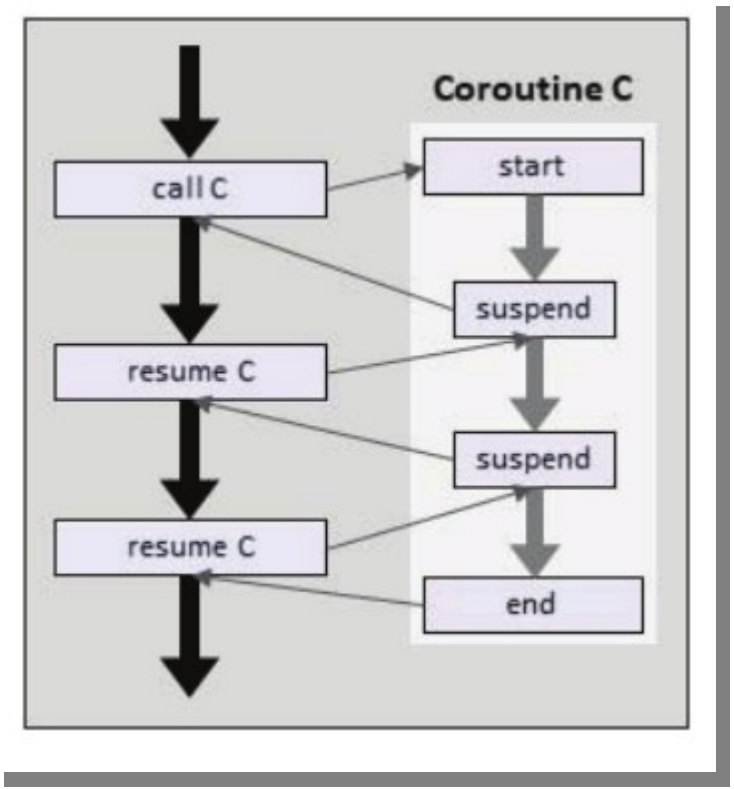




Function / Routine

Function	Coroutine
starts	starts
ends	ends
	suspend
	resume

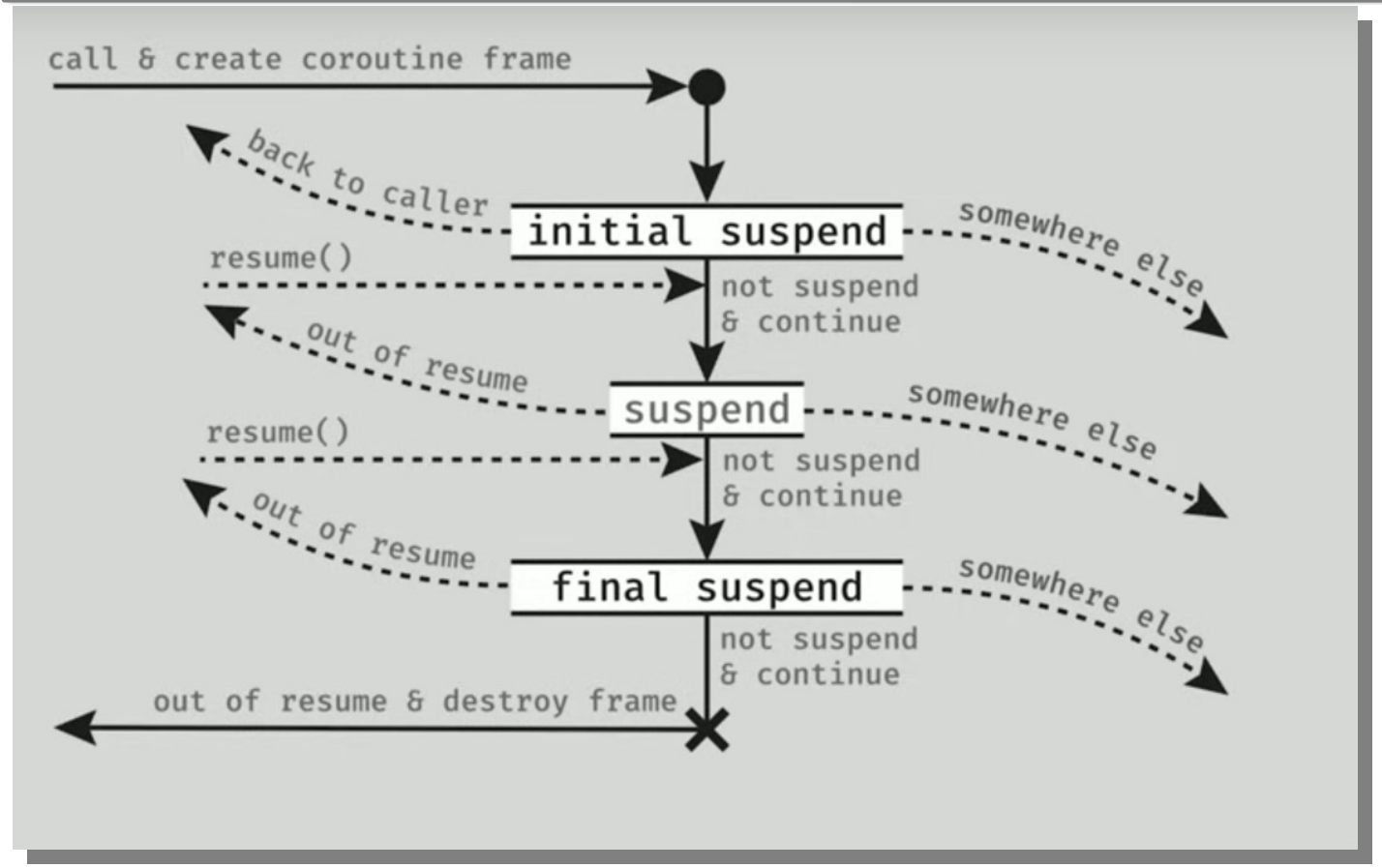
Flow



- Returns to the (same) caller
- Same thread (unless ….)

- Image : (c) Nicolai Josuttis

Advanced flow



- Image : (c) Hana Dusíková

coroutine

- auto foo(…) { … }
  - Is a coroutine when the body contains either:
    - co_return
    - co_yield
    - co_await
  - From that moment the compiler treats it as a coroutine and the magic happens

2 sides to the story

- The compiler facing side
- The user facing side

First example

Let’s try a first example

Radio Station

- At set-up/construction:
  - Type of music
  - How many songs
- A radio station / show emerges, which plays songs
- Modeled by a coroutine
  - Yields a new song and suspends
  - After the user listened to the song, it gets resumed (aka press next)

Song database

```
const int Songs{4};
const std::array<std::string, Songs> electronic
{
    "Front 242 -> No Shuffle",
    "The Neon Judgement -> Tomorrow in the papers",
    "Orbital -> Chime",
    "Underworld -> Born Slippy"
};
const std::array<std::string, Songs> rap
{
    "Public Enemy -> Fight The Power",
    "Ice Cube -> It was a good day",
    "Run DMC -> Walk Thiw Way",
    "Beastie Boys -> Sabotage"
};
```

As a regular function

```
void radioStation(int style, int songs)
{
    for (int i = 0; i < songs; ++i)
    {
        const auto idx = i % Songs;
        std::cout << (style ? electronic[idx] : rap[idx]) << std::endl;
    }
}
```

- We get all the songs at the same time, all or nothing
- At the ‘stream-out’ spot we would like to return 1 song, and later on continue where we left of and provide the next song

The interface towards the user

- Create the coroutine → results in some
  - Coroutine Interface
  - Coroutine api
  - Coroutine remote control
  - …. whatever you want to call it
  - …. something we can further interact with
- Ask for the next song and get it
- Check if there are more songs
- Stop / clean up/ time to go home

This interface sounds more like an object, rather than a function.

So a function (coroutine) needs to result in this “object”.

The function (coroutine has state)

- Let’s assume the music database is a constant global container
- Further (dynamic) state:
  - The type of music
  - The number of songs to play
  - The index in the list of songs (where are we, what’s next, …)
- When suspended, this needs to be stored away
- When resumed, this needs to be restored
- **Stack less** (versus stack full) coroutines
  - The stack frame is stored on the heap by the coroutine mechanism
  - Which as such contains the state described above

Stack frame

- When a function is called some stuff happens
  - Put the return address on the stack
  - Return value (in case not void) placeholder
  - Put the function call arguments on the stack
  - Inside the function : local variables are put on the stack
- When the function returns
  - All the above is cleaned up
- The caller continues in its stack frame, and the callee’s stack stuff is gone, as if it was never there

Stack Frame

- We can’t get rid of the stack frame the moment the co-routine yields in the middle
- We need to resume where we left of
- ===> stack frame needs to be restored at that time
- This means:
  - Stack frame needs to be stored somewhere safe (typically on the heap)
  - It needs to be cleaned up at the correct time
    - This is the programmers responsibility !

Stack Frame => coroutine handle

- Stack frame gets put in some allocated memory
- We get a pointer/**handle** to it
  - Which we need to hold on to
  - Destroy at the appropriate time
    - So at some point during the construction of the co-routine we will get this handle from the eco-system
      - We will come back to this later on

As a couritine

```
RadioStation radioStation(int style, int songs)
{
    for (int i = 0; i < songs; ++i)
    {
        const auto idx = i % Songs;
        co_yield( style ? electronic[idx] : rap[idx] );
    }
}
```

- Spot the 2 differences …
  - co_yield
  - Return Value → RadioStation ===> ‘that coroutine object’
    Note: there is no return statement, so we don’t return a regular value (so we return void), => we return object and return nothing … as we will see later we can also return a regular value instead of void (co_return)

RadioStation object : user perspective

- Create it → constructor → the user is NOT directly calling it
- Destroy it → destructor
- Are we done ? → done()
- Next song → next()
  - Obviously not to be called when we are done
  - Coroutine resumes, we get the next song and the coroutine suspends
- Eager/Lazy
  - When created, do we get immediately a song
    - Yes : Eager
    - No : Lazy

We go for lazy, we consider the creation, like plugging the radio into the power socket, we don't turn it on yet (aka play song), so we always explicitly ask for the (first/next) song

RadioStation object : user perspective

- next and done ==> nextSong()
  - Returns true : there is more
  - Returns false : it’s done
- So after resumption we only get signalled, if there was something more
- So need to explicitly fetch our new song

User perspective : looks like

```
int main()
{
    auto radio = radioStation(0, 7);
    while (radio.nextSong())
    {
        std::cout << radio.value() << std::endl;
    }
    return 0;
}
```

Compiler perspective

- Needs to store (computed/return) values
- Needs some configuration:
  - Suspend at startup (eager/lazy)
  - Suspend at the end
  - What to do if an exception occurs
  - Create the “return object”, aka the “api”, aka our RadioStation object
    - When it creates this object it needs to pass in the coroutine handle → constructor argument
- All this is done via the “promise_type”, aka an object (eg. a struct) which adheres to the **promise_type concept**, it lives inside the coroutine handle → coroutine handle is templatized on this promise_type
- promise_type ==> needs to provide *certain* methods, not all possible methods of the concept
- Has nothing to do with std::promise

promise_type : startup : suspend ?

- auto initial_suspend()
  - Yes

```
auto initial_suspend()
{
    return std::suspend_always{};
}
```

  - No

```
auto initial_suspend()
{
    return std::suspend_never{};
}
```

promise_type : at end : suspend ?

- auto final_suspend()
  - Yes

```
auto final_suspend()
{
        return std::suspend_always{};
}
```

  - No

```
auto final_suspend()
{
        return std::suspend_never{};
}
```

promise_type : at exception ?

- void unhandled_exception()

- ```
  void unhandled_exception() noexcept
  {
      std::terminate();
  }
  ```
  ```

promise_type : current state of knowledge

```
auto initial_suspend()
{
    return std::suspend_always{};
}

void unhandled_exception()
{
    std::terminate();
}

auto final_suspend() noexcept
{
    return std::suspend_always{};
}
```

promise_type : return at end

- What will the coroutine return when the closing brace is hit
  - Nothing (current case, aka return void)
  - Something (see later)
- Compiler needs to know this
  - return_void()
  - return_value(...)

promise_type : return at end

- We do need to implement another method in the promise_type: **return_void**
  - Nothing special we want to do here → empty implementation
    void return_void() {}

promise_type : current state of knowledge

```
auto initial_suspend()
{
    return std::suspend_always{};
}

void unhandled_exception()
{
    std::terminate();
}

auto final_suspend() noexcept
{
    return std::suspend_always{};
}
void return_void() {}
```

promise_type : store value

- When the coroutine **GENERATES** values (co_yield or co_return), it needs to be able to store them somewhere for the user later on to retrieve ==> in the promise_type
- And when this value is provided → suspend : yes/no ?
- auto yield_value(const T& value)

promise_type : store value

```
auto yield_value(const std::string& valueIn)
{
    value = valueIn;
    return std::suspend_always{};
}
std::string value;
```

promise_type : current state of knowledge

```
auto initial_suspend()
{
    return std::suspend_always{};
}
void unhandled_exception()
{
    std::terminate();
}
auto final_suspend() noexcept
{
    return std::suspend_always{};
}
void return_void() {}
auto yield_value(const std::string& valueIn)
{
     value = valueIn;
     return std::suspend_always{};
}
std::string value;
```

promise_type : still MISSING

- auto get_return_object()
  - We will come back to this shortly

Hierarchy

- promise_type
  - Is stored in the coroutine HANDLE
- Handle
  - Is stored in the ‘return object/api’ aka RadioStation
- The return object gets the handle at constructor time (so it does not create it), and then owns it and (might need) to destroy it

Handle

- Templatized on the promise_type
  - std::coroutine_handle<promise_type>
- Provides methods to interact with:
  - resume()
  - done()
  - destroy()
  - Is there (still) a handle → if(handle)
  - promise()

RadioStation : return object

```
1  class [[nodiscard]] RadioStation
2  {
3  public:
4      struct promise_type;
5      using CoroHandle = std::coroutine_handle<promise_type>;
6
7
8      RadioStation(auto handle) :
9          mHandle{handle}
10     {
11     }
12
13
14     ~RadioStation()
15     {
16         if (mHandle)
17         {
18             mHandle.destroy();
19         }
20     }
21
22     // …. more ...
23
24 private:
25     CoroHandle mHandle;
26 };
```

RadioStation : done or do next

```
bool nextSong() const          /// you could call it: resume()
{
    if (!mHandle || mHandle.done())
    {
        return false;
    }
    mHandle.resume();
    return !mHandle.done();
}
```

RadioStation : done or do next

```
bool nextSong() const
{
    return mHandle ?
            (mHandle.resume(), !mHandle.done())
                : false;
}
```

RadioStation : done or do next

- Bool return value
  - True : there is more
  - False : nothing left to do, don’t call us anymore, it was nice meeting you

RadioStation : get the value

```
std::string value() const
{
    return mHandle.promise().value;
}
```

promise_type : get_return_object

- auto get_return_object()
  - Called by the compiler to create the return object
  - Needs to accept at constructor time the coroutine handle
  - And whatever other constructor arguments we came up with
- Multiple steps
  1) Create the promise_type
  2) Create the coroutine handle
  3) Create the real return object (Eg. RadioStation)
  4) And return it

promise_type : get_return_object

- Step 1 and Step 2 can be combined
- std::coroutine_handle<promise_type>::from_promise(*this)

```
auto get_return_object()
{
        return RadioStation{CoroHandle::from_promise(*this)};
}
```

Putting it all together : RadioStation class

```
class [[nodiscard]] RadioStation
{
public:
    struct promise_type;
    using CoroHandle = std::coroutine_handle<promise_type>;

    RadioStation(auto handle) :
        mHandle{handle}
    {
    }
    ~RadioStation()
    {
        if (mHandle)
        {
            mHandle.destroy();
        }
    }
    RadioStation(const RadioStation&) = delete;
    RadioStation& operator=(const RadioStation&) = delete;

    bool nextSong() const
    {
        if (!mHandle || mHandle.done())
        {
            return false;  // we are done
        }
        mHandle.resume();
        return !mHandle.done();
    }


    std::string value() const
    {
        return mHandle.promise().value();
    }
private:
    CoroHandle mHandle;
};
```

Putting it all together : promise_type

```
77      struct promise_type
78      {
79          auto get_return_object()
80          {
81              return RadioStation{CoroHandle::from_promise(*this)};
82          }
83
84          auto initial_suspend()
85          {
86              return std::suspend_always{};
87          }
88
89          void unhandled_exception()
90          {
91              std::terminate();
92          }
93
94          auto yield_value(const std::string& valueIn)
95          {
96              value = valueIn;
97              return std::suspend_always{};
98          }
99
100         void return_void() { }
101
102         auto final_suspend() noexcept
103         {
104             return std::suspend_always{};
105         }
106
107         std::string value;
108     };
```

Putting it all together : use it

```
RadioStation radioStation(int style, int songs)
{
    for (int i = 0; i < songs; ++i)
    {
        const auto idx = i % Songs;
        co_yield( style ? electronic[idx] : rap[idx] );
    }
};
```

```
int main()
{
    auto radio = radioStation(0, 7);
    while (radio.nextSong())
    {
        std::cout << radio.value() << std::endl;
    }
    return 0;
}
```

I want my ~~MTV~~ iterator

- In main we would like the user code to look like

```
auto radio = radioStation(0, 7);

for (const auto& song : radio)
{
    std::cout << song << std::endl;
}
```

I want my ~~MTV~~ iterator

- we need begin()/end() which gives us an iterator
- iterator needs to be able to be :
  - incremented : operator++
  - dereferenced : operator*
- iterators need to be able to be compared (to end iterator)
- the iterator on dereferencing needs to provide the value
- the iterator on increment means 'resume'
- Our Iterator type will be inside RadioStation

===> it will need both the handle and the promise (which we get via the handle)

===> iterator needs to get the handle at constructor time, and store it

===> say when done we remove the stored handle from the iterator (aka nullptr)

I want my ~~MTV~~ iterator

```
112     struct Iterator
113     {
114         Iterator(auto p) : mHandle{p} {}
115
116         std::string operator*()
117         {
118             return mHandle.promise().value;
119         }
120
121         Iterator operator++()
122         {
123             if(mHandle)
124             {
125                 mHandle.resume();
126                 if(mHandle.done())
127                 {
128                     mHandle = nullptr;
129                 }
130             }
131             return *this;
132         }
133
134         bool operator== (const Iterator&) const = default;
135
136         std::coroutine_handle<promise_type> mHandle;
137     };
```

I want my ~~MTV~~ iterator

- The 2 new methods on the RadioStation class

```
139     Iterator begin() const
140     {
141         if(!mHandle || mHandle.done())
142         {
143             return Iterator{nullptr};
144         }
145         Iterator iter{mHandle};
146         return ++iter;
147     }
148
149     Iterator end() const
150     {
151         return Iterator{nullptr};
152     }
```

Conclusion till now

- Coroutine function itself : easy
- User code in main : easy
- All the boilerplate concepts:
  - Not that hard
  - Annoying we should deal with it
- Is this production ready ?
  - We are the experts, right ?
    - Is your colleague also ?
- If we just would have … (see next slide)

Name dropping

- We have a **GENERATOR**, generating std::string's
- C++23 : std::generator

```
using RadioStation = std::generator<std::string>;

RadioStation radioStation(int style, int songs)
{
    for (int i = 0; i < songs; ++i)
    {
        const auto idx = i % Songs;
        co_yield( style ? electronic[idx] : rap[idx] );
    }
}
```

Second example

Let’s try a second example



A co-routine which does not yield/return anything

- So the co-routine does some work on each resumption
- Might never stop
- **co_await**

- Stupid example: 2 – players – pinball
  - The caller is 1 player (I)
  - The co-routine is the other player (You)
  - We each take turns, we **collaboratively** work on the shared pinball machine
  - We want to win, so we will make the co-routine loose after N turns, aka the co-routine will be done

A co-routine which does not yield/return anything

- The interface/api:
  - Resume method, let’s call it play()
  - No need for a get value method, since it returns/yields nothing
  - We configure N (turns) by specifying an integer to the co-routine invocation

Putting it all together : use it

```
Pinball pinball(int turns)
{
    for (int i = 0; i < turns; ++i)
    {
      std::cout << “Coro’s turn to play.\n”;
      co_await std::suspend_always{};
    }
    std::cout << “Coro looses.\n”;
};
```

```
int main()
{
    auto pball = pinball(4);
    while (pball.play())
    {
        std::cout << “My turn to play.\n”;
    }
    return 0;
}
```

promise_type

- There’s no need to store anything
  - So no member needed
  - No yield_value() needed

Putting it all together : promise_type

```
66  struct Pinball::promise_type
67  {
68      auto get_return_object()
69      {
70          return Pinball{CoroHandle::from_promise(*this)};
71      }
72
73      auto initial_suspend()
74      {
75          return std::suspend_always{};
76      }
77
78      void unhandled_exception()
79      {
80          std::terminate();
81      }
82
83      void return_void()
84      {
85      }
86
87      auto final_suspend() noexcept
88      {
89          return std::suspend_always{};
90      }
91  };
```

Putting it all together : Pinball class

```
class [[nodiscard]] Pinball
{
public:
    struct promise_type;
    using CoroHandle = std::coroutine_handle<promise_type>;

    Pinball(auto handle) :
        mHandle{handle}
    {}
    ~Pinball()
    {
        if (mHandle)
        {
            mHandle.destroy();
        }
    }
    Pinball(const Pinball&) = delete;
    Pinball& operator=(const Pinball&) = delete;

    bool play() const
    {
        if (!mHandle || mHandle.done())
        {
            return false;  // we are done
        }
        mHandle.resume();
        return !mHandle.done();
    }
private:
    CoroHandle mHandle;
};
```

Cheat the system

- C++23 : std::generator

```
using Pinball = std::generator<int>;
Pinball pinball(int turns)
{
    for (int i = 0; i < turns; ++i)
    {
        std::cout << "      Your turn to play.\n";
        co_yield 0;
    }
    std::cout << "      You loose.\n";
}
```

```
int main()
{
    auto pball = pinball(4);
    for(const auto& turn : pinball)
    {
        (void)turn;
        std::cout << "My turn to play.\n";
    }
    std::cout << "I win.\n";
    return 0;
}
```

Third example

Let’s try a third example

A co-routine which only returns something at the end, when it’s done

- So the co-routine does some work on each resumption
- And at the very end the outcome is ready and returned
- **co_await , co_return**

- Stupid example: calculating the average of a very large set of values
  - Per resumption the co-routine processes a certain amount of the data
  - After several resumptions all the work is done
  - And the result is available and can be returned

A co-routine which only returns something at the end, when it’s done

- The interface/api:
  - Resume method, let’s call it calculate()
  - Again need for a ‘get value’ method, to retrieve the final outcome, let’s call it getResult()

Putting it all together : use it

```
Average average(const std::vector<int>& numbers)
{
    int sum{};
    for (const auto& number : numbers)
    {
        std::cout << "      Number crunching the next value.\n";

        sum += number;
        co_await std::suspend_always{};
    }
    std::cout << "      Finally, all calculated.\n";
    co_return sum / numbers.size();
};
```

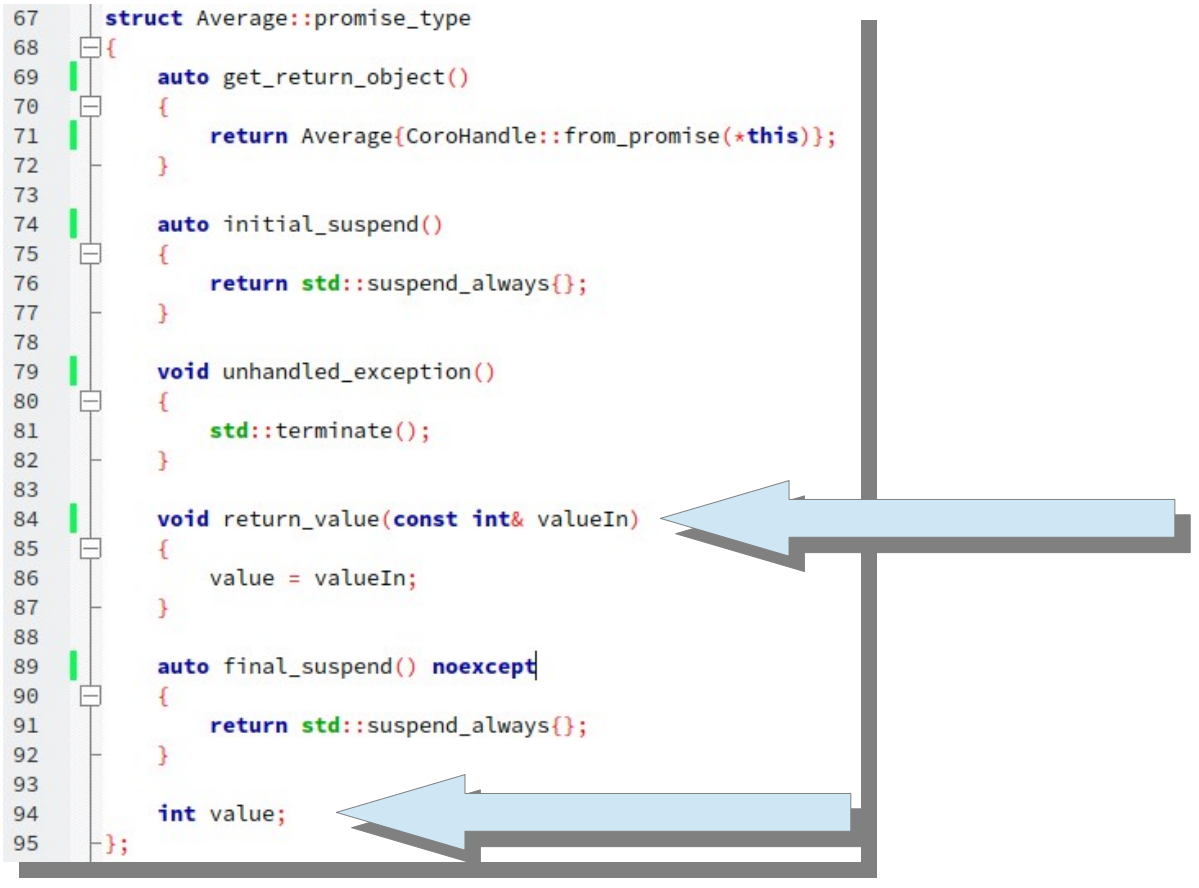
```
int main()
{
    const std::vector<int> numbers{100, 200, 100, 200, 100, 200, 100, 200};
    auto aver = average(numbers);

    while (aver.calculate())
    {
        std::cout << "More calculations needed.\n";
    }
    std::cout << "Average : " << aver.getResult() << std::endl;
    return 0;
}
```

promise_type

- Again need to store : the end value
- We do need for that, to implement another method in the promise_type to allow to store the value: **return_value**
- promise_type as such again has a **member variable**, to be filled in during that return_value(…) call

Putting it all together : promise_type



```
67  struct Average::promise_type
68  {
69      auto get_return_object()
70      {
71          return Average{CoroHandle::from_promise(*this)};
72      }
73
74      auto initial_suspend()
75      {
76          return std::suspend_always{};
77      }
78
79      void unhandled_exception()
80      {
81          std::terminate();
82      }
83
84      void return_value(const int& valueIn)
85      {
86          value = valueIn;
87      }
88
89      auto final_suspend() noexcept
90      {
91          return std::suspend_always{};
92      }
93
94      int value;
95  };
```

Putting it all together : Average class

```
class [[nodiscard]] Average
{
public:
    struct promise_type;
    using CoroHandle = std::coroutine_handle<promise_type>;

    Average(auto handle) :
        mHandle{handle}
    { }
    ~Average()
    {
        if (mHandle)
        {
            mHandle.destroy();
        }
    }
    Average(const Average&) = delete;
    Average& operator=(const Average&) = delete;

    bool calculate() const
    {
        if (!mHandle || mHandle.done())
        {
            return false;  // we are done
        }
        mHandle.resume();
        return !mHandle.done();
    }

    int getResult() const
    {
        return mHandle.promise().value;
    }

private:
    CoroHandle mHandle;
};
```

Cheat the system

- C++23 : std::generator

```
6   namespace
7   {
8
9   using Average = std::generator<std::optional<int>>;
10
11  Average average(const std::vector<int>& numbers)
12  {
13      int sum{};
14      for (const auto& number : numbers)
15      {
16          std::cout << "      Number crunching the next value.\n";
17
18          sum += number;
19          co_yield {};
20      }
21      std::cout << "      Finally, all calculated.\n";
22      co_yield sum / numbers.size();
23  }
24  } // namespace
25
```

```
26  int main()
27  {
28      const std::vector<int> numbers{100, 200, 100, 200, 100, 200, 100, 200};
29      auto aver = average(numbers);
30
31      for ( const auto& res : aver)
32      {
33          if(res)
34          {
35              std::cout << *res << '\n';
36          }
37      }
38      return 0;
39  }
```

Wait wait wait

Awaiters/Awaitable

Wait a minute

- Awaitables → the operand of co_await
- Awaiters → specific way to implement an awaitable

- Is another configuration point
- Used whenever co_await and co_yield is called

Awaiter

- 3 methods
  - await_ready()
  - await_suspend(awaitHandle)
  - await_resume()

await_ready

- Called immediately **before** the coroutine is suspended
- Allows as such, for some reason, to decide not to suspend after all
- Returns true → coroutine is NOT suspended
- Typically : return false;
- Use case : suspension depends on some data availability

await_suspend(awaitHandle)

- Called immediately **after** the coroutine is suspended
  - But before control is handed back to the caller
- Parameter : the handle of the coroutine that was suspended
- What to do next ? Again we could neutralize the suspension if we want. We could even destroy the coroutine …

auto await_resume()

- Called when the coroutine is resumed (after a successful suspension)
- Can return a value



  - The value the co_await expression yields

2 Awaiters we know

- std::suspend_always
  - await_ready returns false
- std::suspend_never
  - await_ready returns true

- The 2 other methods are empty, and await_resume returns void
- Aka both do nothing at suspension and resumption

Application : coroutines calling coroutines

- See later

Application : passing values from suspension back to co-routine

- When the co-routine yields us a value at suspension time, we would like from the caller side, pass in a value ourselves for the next resumption cycle
- Example:
  - When we get the song in our RadioStation
  - We will provide a score of our appreciation of this song when requesting(resuming) the next song

Mechanisms we already know

- Passing values between the 2 sides => promise_type
  - So new member in the promise_type
  - In the API a method for the user to pass in this value
    - The implementation will then store it in the promise_type
- How does the co-routine get's it out of the promise_type ?
  - Return value of co_yield call
  - Custom Awaiter
- Let's inspect these 1 by 1

Api change

- Method needed to pass our value
- Let’s call it : score(…)
- Stores it in the promise_type

```
void RadioStation::score(int scoreIn)
{
    mHandle.promise().score = scoreIn;
}
```

Api change

- Calling it

```
int main()
{
    auto radio = radioStation(0, 7);

    while (radio.nextSong())
    {
        std::cout << radio.value() << std::endl;
        radio.score(10);
    }
    return 0;
}
```

promise_type change

- Member needed
- Different **yield_value** (see later)

```
struct RadioStation::promise_type
{
...

    std::string value;
    int score{};
};
```

Co-routine method

- Get’s the value as return value of the co_yield call

```
for (int i = 0; i < songs; ++i)
{
   const int score = co_yield( style ? electronic[track(rng)] : rap[track(rng)] );
   std::cout << "The previous track scored " << score << std::endl;
}
```

promise_type change

- We used to return the Awaiter std::suspend_always in yield_value(...)
- Now we return an instance of our home made Awaiter (template parameter will be explained shortly)

```
auto yield_value(const std::string& valueIn)
{
    value = valueIn;
    return MyAwaiter<CoroHandle>{};
}
```



Awaiter : remember

- await_resume can return a value
  - The value the co_await expression yields
- So that means in this case our score
  - Which is stored in the promise_type
  - Accessible via the handle
- Let’s assume the handle is a member of our Awaiter, we will see later how it ended up in there

Awaiter

```
template <typename Handle>
struct MyAwaiter
{
    Handle hdl{nullptr};

    bool await_ready() const noexcept
    {
        return false;
    }

/// 1 method missing, see later

    auto await_resume() const noexcept
    {
        return hdl.promise().score;
    }
}
```



Awaiter : remember

- await_suspend receives as argument the handle of the co-routine that has just been suspended
  - So it can store it
  - And await_resume can use it

Awaiter

```
82  template <typename Handle>
83  struct MyAwaiter
84  {
85      Handle hdl{nullptr};
86
87      bool await_ready() const noexcept
88      {
89          return false;
90      }
91
92      void await_suspend(Handle hdlIn) noexcept
93      {
94          hdl = hdlIn;
95      }
96
97      auto await_resume() const noexcept
98      {
99          return hdl.promise().score;
100     }
101 };
```



Awaiter



Coroutine calling another coroutine

- Say outer coroutine suspends at certain points
- Somewhere it calls another coroutine
  - We want the inner suspensions to be like it were suspensions of the outer
-

...

coroInner()

...

/// this does not work !

Coroutine calling another coroutine

- We need to explicitly LOOP

```
MyCoro coroOuter()
{
    std::cout << "\t\t coroutine Outer started, calling the inner one" << "\n";

    auto sub = coroInner(3);
    while (sub.resume())
    {
        std::cout << "  Outer(): coroInner() suspended\n";
    }
    std::cout << "\t\t outer done with calling the inner, time to suspend outer\n";
    co_await std::suspend_always{};
    std::cout << "\t\t outer done\n";
}
```

Coro call coro : Awaitable

- Have the API/Object of the calling coroutine be an Awaitable
  - This allows : **co_await coroInner(3)**
- With some more boilerplate the suspend/resume of the inner will be to the outside world as if they were from the outer one
- Remember:

```
bool resume() const
{
    if (!mHandle || mHandle.done())
    {
        return false;  // we are done
    }
    mHandle.resume();
    return !mHandle.done();
}
```



Coro call coro : Awaitable

- What if ?
  - We would resume the handle from the inner one, from within the resume method of the outer
  - Requires:
    - Outer knowing there is an inner handle
    - Checking if inner done or not, if not, resume that handle, if done, resume own (outer) handle
  - Assume we were able to store that handle in the promise_type (of the Outer) : mSub

Coro call coro : Awaitable

```
bool resume() const
{
    if (!mHandle || mHandle.done())
    {
        return false;
    }

    CoroHandle handle{mHandle};
    while(handle.promise().mSub &&
        ! handle.promise().mSub.done() )
    {
        handle = handle.promise().mSub;
    }
    handle.resume();
    return !mHandle.done();
}
```

Coro call coro : Awaitable

- How to get that sub handle in the promise_type of the Outer ?
- Remember Inner is an Awaitable (handle : mHandle)
  - **void await_suspend(auto awaitHdl)**
- This means that “co_await coroInner(3);” :
  - Will call the Awaiter (Inner coroutine object)
  - Passing in the handle of the coroutine that got suspended (handle of Outer)
    - Inner now knows the handle of Outer
    - Fetch the promise from it
    - Store it’s own handle in it

Coro call coro : Awaitable

```
void await_suspend(auto awaitHdl)
{
    awaitHdl.promise().mSub = mHandle;
}
```

Coroutine calling another coroutine

```
 93  MyCoro coroInner(int maxIterations)
 94  {
 95      std::cout << "\t\t\t coroutine Inner started for at max iterations:  "
 96                << maxIterations << "\n";
 97
 98      for (int val = 1; val <= maxIterations; ++val)
 99      {
100          std::cout << "\t\t\t Iteration " << val << '/' << maxIterations << '\n';
101
102          co_await std::suspend_always{};
103      }
104
105      std::cout << "\t\t\t all iterations done, coroutine ends\n";
106  }
107
108  MyCoro coroOuter()
109  {
110      std::cout << "\t\t coroutine Outer started, calling the inner one" << "\n";
111
112      co_await coroInner(3);
113
114      std::cout << "\t\t outer done with calling the inner, time to suspend outer\n";
115      co_await std::suspend_always{};     // SUSPEND
116      std::cout << "\t\t outer done\n";
117  }
```



Coro call coro : Awaitable



co_await (expression) : more

- Can be an expression resulting in any type (say Foo)
- Requires either:
  - The promise_type has a method: **auto await_transform(Foo x)**
  - Foo has : **auto operator co_await()**
- Both scenarios need to return a real Awaiter

```
co_await 242;

co_await Foo{};
```

await_suspend : return type

- There can be 3 different return types
  - void
  - bool
  - handle

await_suspend : return void

- We can do some extra stuff
- Suspension remains in place

await_suspend : return bool

- We can do some extra stuff
- Suspension remains in place when true is returned, canceled when false is returned

await_suspend : return coroutine handle

- We can do some extra stuff
- The handle of this (other) coroutine is resumed
- Symmetric transfer
- std::noop_coroutine() is case no other coroutine handle to return
- **CONTINUATION**

Some day : std::future::then

- std::async → std::future
- When the async is done ‘then’ do the next thing:
  - Callable passed to myFuture**.then**(...)
- **CONTINUATION**

Boost asio : completion handler

- When the async operation is finished, the completion handler (token) is executed
- **CONTINUATION**

```
boost::asio::async_read(
    client->mSocket,
    boost::asio::dynamic_buffer(client->mReadBuffer),
    std::bind(completionCondition, std::ref(client->mReadBuffer), std::placeholders::_1,
std::placeholders::_2),
    std::bind(&Server::readHandler, this, client, std::placeholders::_1, std::placeholders::_2));
```

Continuation

- Idea:
  - CoroOuter calls coroInner
  - When inner is ‘finished’, outer should continue, aka outer is the continuation of inner
    - When inner finished
    - Then continue with outer

Continuation

- How:
  - An Awaiter with await_suspend returning the handle of outer
  - An Awaiter to be invoked from inner, at the ‘appropriate time’
    - When it is finishing
      - ==> auto **final_suspend**()

Teaser : boost::asio and coroutines

```cpp
boost::asio::awaitable<void> echo(
        boost::asio::ip::tcp::socket peer_socket,
        boost::asio::ip::tcp::acceptor acceptor)
{
    std::array<char, 1000> buf;

    for (;;)
    {
        co_await acceptor.async_accept(peer_socket, boost::asio::use_awaitable);

        for (;;)
        {
            const auto [error, len] = co_await peer_socket.async_read_some(
                boost::asio::buffer(buf),
                boost::asio::as_tuple(boost::asio::use_awaitable));
            if (error == boost::asio::error::eof)
            {
                break;
            }

            co_await async_write(
                peer_socket,
                boost::asio::buffer(buf, len),
                boost::asio::use_awaitable);
        }
        peer_socket.close();
    }
}
```

```cpp
boost::asio::io_context ctx;


boost::asio::ip::tcp::socket socket{ctx};
boost::asio::ip::tcp::acceptor acceptor{ctx,
        boost::asio::ip::tcp::endpoint{boost::asio::ip::tcp::v4(), 6666}};


boost::asio::co_spawn(
        ctx,
        echo(std::move(socket), std::move(acceptor)),
        boost::asio::detached);


ctx.run();
```

QUESTIONS